



Process Synchronization(1)

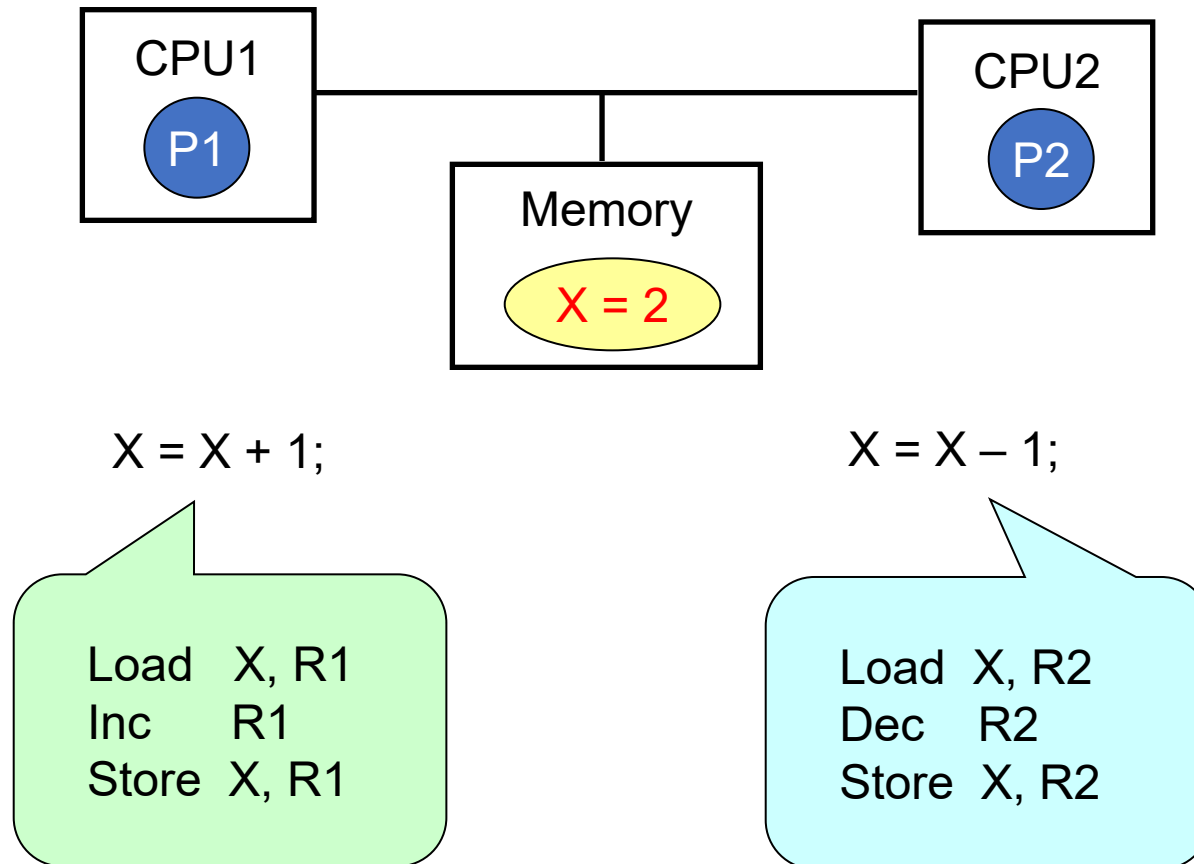
Dept. of Computer Science
Hanyang University





- *Concurrent access* to *shared data* may result in data *inconsistency*
- Maintaining data consistency requires mechanisms to ensure the *orderly execution* of cooperating processes
- **Race condition**
 - The situation where several processes access and manipulate shared data concurrently
 - The final value of the shared data depends upon which process finishes last
- To prevent race conditions,
concurrent processes must be *synchronized*

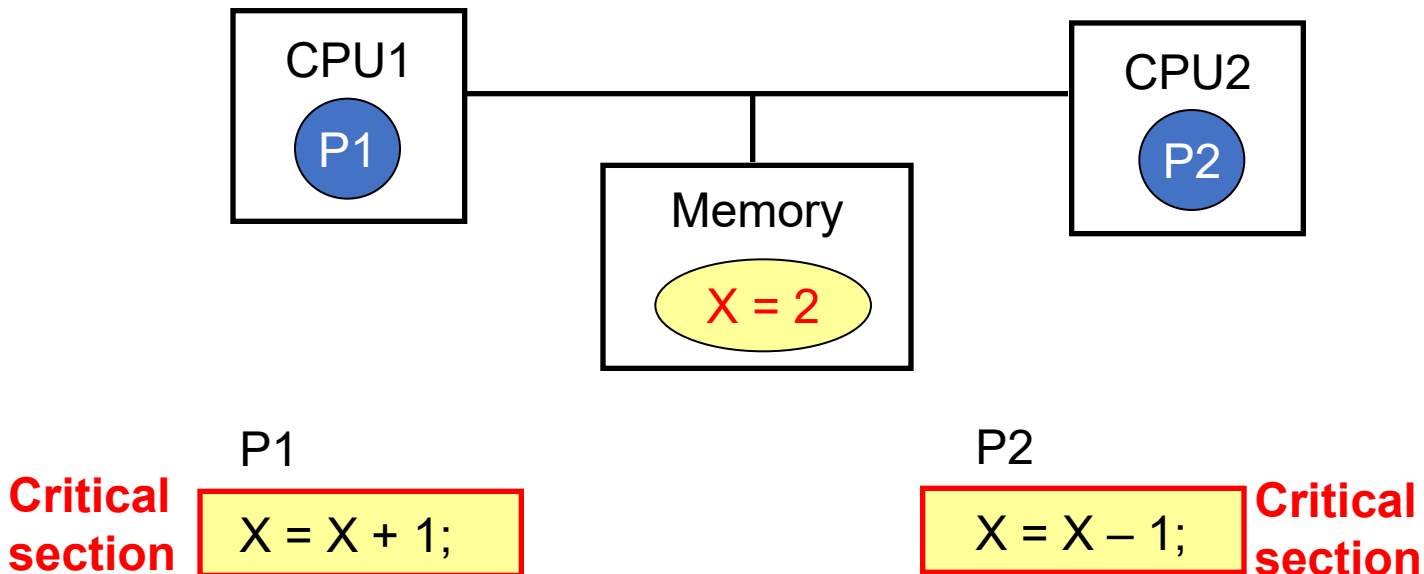
Example of a Race Condition



What happens if the instructions are interleaved?

The Critical-Section Problem

- n processes all competing to use some shared data
- Each process has a code segment, called **critical section**, in which the *shared data is accessed*
- Problem
 - Ensure that *when one* process is executing in its *critical section*, *no other process* is allowed to execute in its critical section



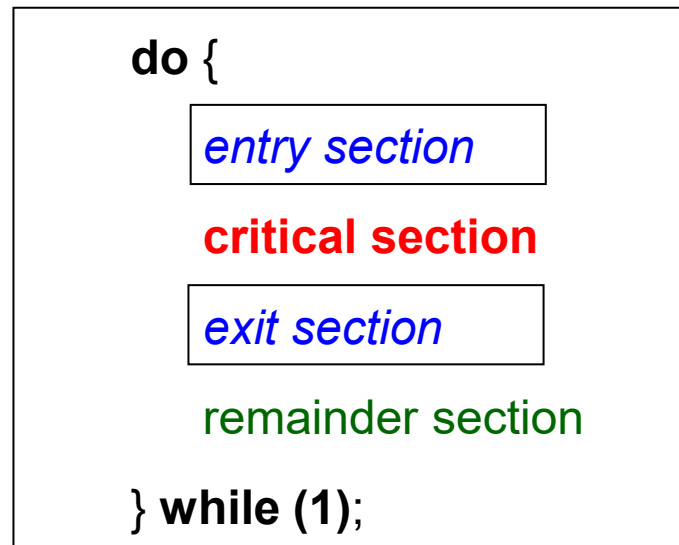


Requirements for the Solution

- **Mutual Exclusion** *(prevent simultaneous execution)*
If process P_i is executing in its critical section, then
no other processes can be executing *in their critical sections*
- **Progress** *(prevent indefinite postponement)*
If no process is executing *in its critical section* and there exist some processes that wish to enter their critical section, then the selection of the process that will enter the critical section next *cannot be postponed indefinitely*
- **Bounded Waiting** *(prevent excessive blocking)*
A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted
 - Assume that each process executes at a nonzero speed
 - No assumption concerning relative speed of the processes

Initial Attempts to Solve Problem

- Only two processes, P_0 and P_1
- General structure of process P_i (other process: P_j)



- Processes may share some common variables to synchronize their actions



Algorithm 1

- Shared variables:
 - **int** **turn**;
initially **turn = 0** $\Rightarrow$ P_i can enter its critical section **if** (**turn == i**)
- Process P_0

```

do {
    while (turn != 0) ;    /* My turn? */
    critical section
    turn = 1;              /* Now it's your turn */
    remainder section
} while (1);

```

- Satisfies mutual exclusion, but *not progress*
 - *Swap-turn* : two processes can execute only in turn
 - If one process was designed to execute the critical section more frequently?



Algorithm 2

- Shared variables

- boolean** **flag[2];**
 - initially **flag [i] = flag [j] = false;** **/* no one is in CS */**

- “ P_i ready to enter its critical section” if (**flag [i] == true**)

- Process P_i

```
do {
    flag[ i ] = true;           /* Pretend I am in */
    while (flag[ j ]) ;        /* Is he also in? then wait*/
    critical section
    flag [ i ] = false;        /* I am out now*/
    remainder section
} while (1);
```

- Satisfies mutual exclusion, but *not progress*
 - when **flag[i] = flag[j] = true**

Algorithm 3 (Peterson's Algorithm)

- Combining shared variables of algorithms 1 and 2.
- Process P_i

```
do {  
    flag [ i ]= true;          /* My intention is to enter .... */  
    turn = j;                  /* Set to his turn */  
    while (flag [ j ] and turn == j) ; /* wait only if ...*/  
    critical section  
    flag [ i ] = false;  
    remainder section  
} while (1);
```

- *Meets all three requirements*; solves the critical-section problem for two processes
- Can be extended to support three or more processes, easily
- *Busy Waiting!* (process uses CPU even when it is waiting)



Solution to Critical-section Problem using Locks

```
do {  
    acquire lock  
    critical section  
    release lock  
    remainder section  
} while (TRUE);
```

Problem:

How to ensure that only one process can acquire the lock at a moment?



Synchronization Hardware

- Many systems provide hardware support for critical section code
- Uniprocessors – could disable interrupts
 - Currently running code would execute without preemption
 - Generally too inefficient on multiprocessor systems
 - Operating systems using this not broadly scalable
- Modern machines provide special atomic hardware instructions
 - Atomic = non-interruptible
 - Either test memory word and set value (Test-and-Set)
 - Or swap contents of two memory words (Swap)

Synchronization Hardware

- *Test and modify* the content of a word atomically

[1] return value = target's original value

[2] target = true

```
boolean TestAndSet(boolean &target) {  
    boolean rv = target;    /* rv: return value*/  
    target = true;  
    return rv;  
}
```

Mutual Exclusion with Test-and-Set

- Shared data:

boolean lock = false;

- Process P_i

do {

while (TestAndSet(lock)) ;

critical section

lock = false;

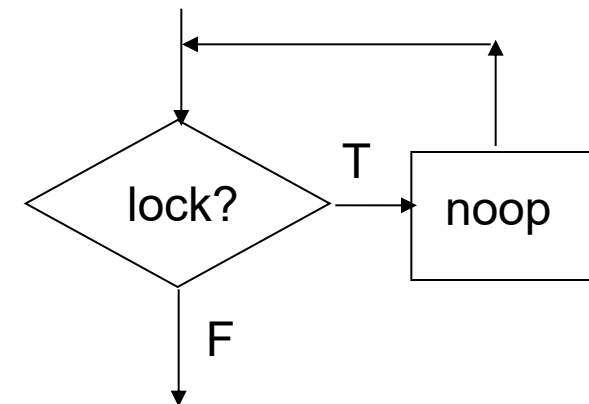
remainder section

}

R.V. TRUE

noop

while(cond) do { };





Synchronization Hardware

- Atomically swap two variables

```
void Swap(boolean &a, boolean &b) {  
    boolean temp = a;  
    a = b;  
    b = temp;  
}
```

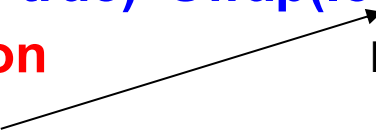
Mutual Exclusion with Swap

- Shared data (initialized to **false**):
boolean lock;

- Process P_i

```
do {                                /* key: me   lock: he */  
    key = true;                    /* My intention */  
    while (key == true) Swap(lock, key);  
    critical section  
    lock = false;  
    remainder section  
}
```

Did he set lock ?



Semaphores

- **Semaphore S**

- integer variable
- can only be accessed via two indivisible (atomic) operations

$P()$: wait function

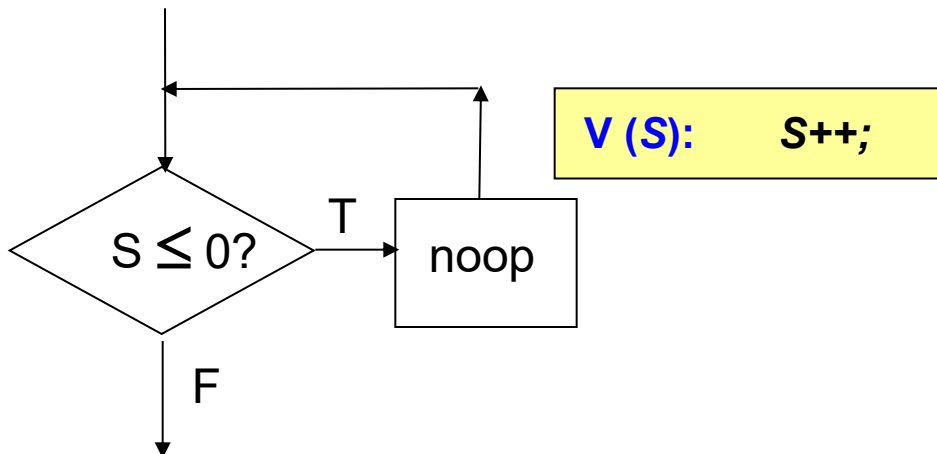
$V()$: signal function

$P(S)$: while ($S \leq 0$) do *no-op* ;
 $S--$;

i.e. wait

If positive, decrement-&-enter.

Otherwise, wait until ++ (busy-wait)

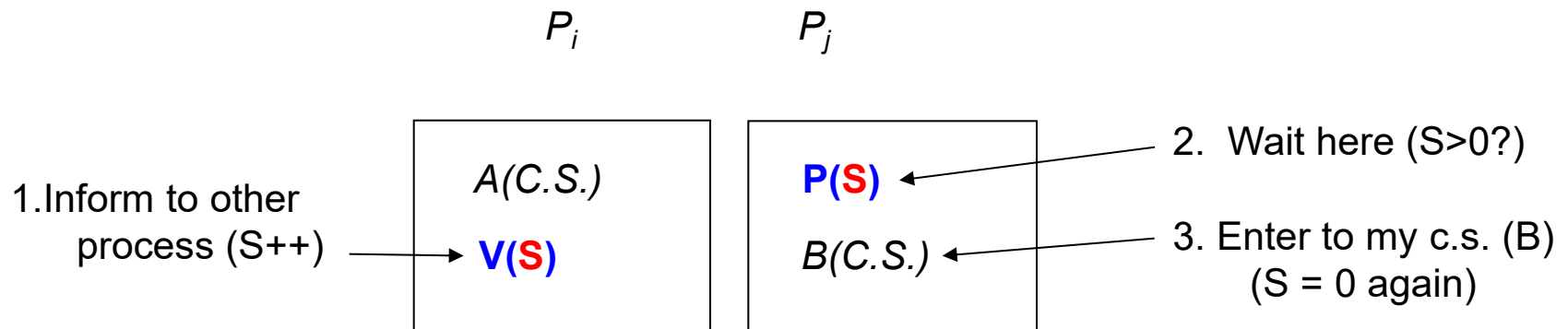


$V(S)$: $S++$;



Semaphore as a General Synchronization Tool

- Two processes and 2 critical sections A & B
- Execute B in P_j only after A executed in P_i
- Use semaphore **S** initialized to 0
- Code:



Here S is binary semaphore - used for mutual exclusion



Critical Section of n Processes

- Shared data:

semaphore mutex; */* initially 1 meaning no one is in CS */*

- Process P_i :

do {

P(mutex); */* If positive, dec-&-enter, Otherwise, wait. */*

critical section

V(mutex); */* Increment semaphore */*

 remainder section

} while (1);



Semaphore Implementation

- Must guarantee that no two processes can execute `wait ()` and `signal ()` on the same semaphore at the same time
 - Thus, implementation becomes the critical section problem where the wait and signal code are placed in the critical section
 - We can use the Peterson's algorithm, for example, to synchronize concurrent accesses to `P()` and `V()`
- Using `busy waiting` in `P()`:
 - Implementation code is short
 - Little busy waiting if critical section rarely occupied
 - Note that applications may spend lots of time in critical sections and therefore this is not a good solution
 - May spend long time in `P()`

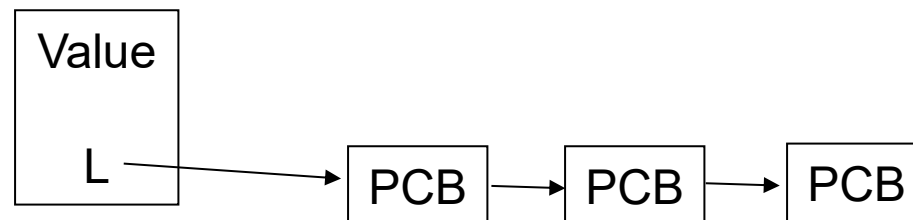
Block / Wakeup Implementation

- Permits multiple processes to enter the wait function
- Define a semaphore as a record

```
typedef struct {
    int          value;      /* semaphore */
    struct process *L;       /* process wait queue */
} semaphore;
```

- Assume two simple operations:
 - **block** kernel suspends the process that invoked itself.
Put this process' PCB into wait queue (semaphore)
 - **wakeup(P)** resumes the execution of a blocked process **P**.
(Put this process' PCB into ready queue)

Semaphore:



Implementation: block/wakeup version of P() & V()

- Semaphore operations now defined as

P(S):

```
S.value--;      /* prepare to enter – i.e. decrement*/  
if (S.value < 0) { /* Oops, negative, I cannot enter*/  
    add this process to S.L;  
    block;  
}
```

V(S):

```
S.value++;  
if (S.value <= 0) { /* absolute value: queue length */  
    remove a process P from S.L;  
    wakeup(P);  
}
```

integer semaphore

value = # of waiting processes
used for process synchronization

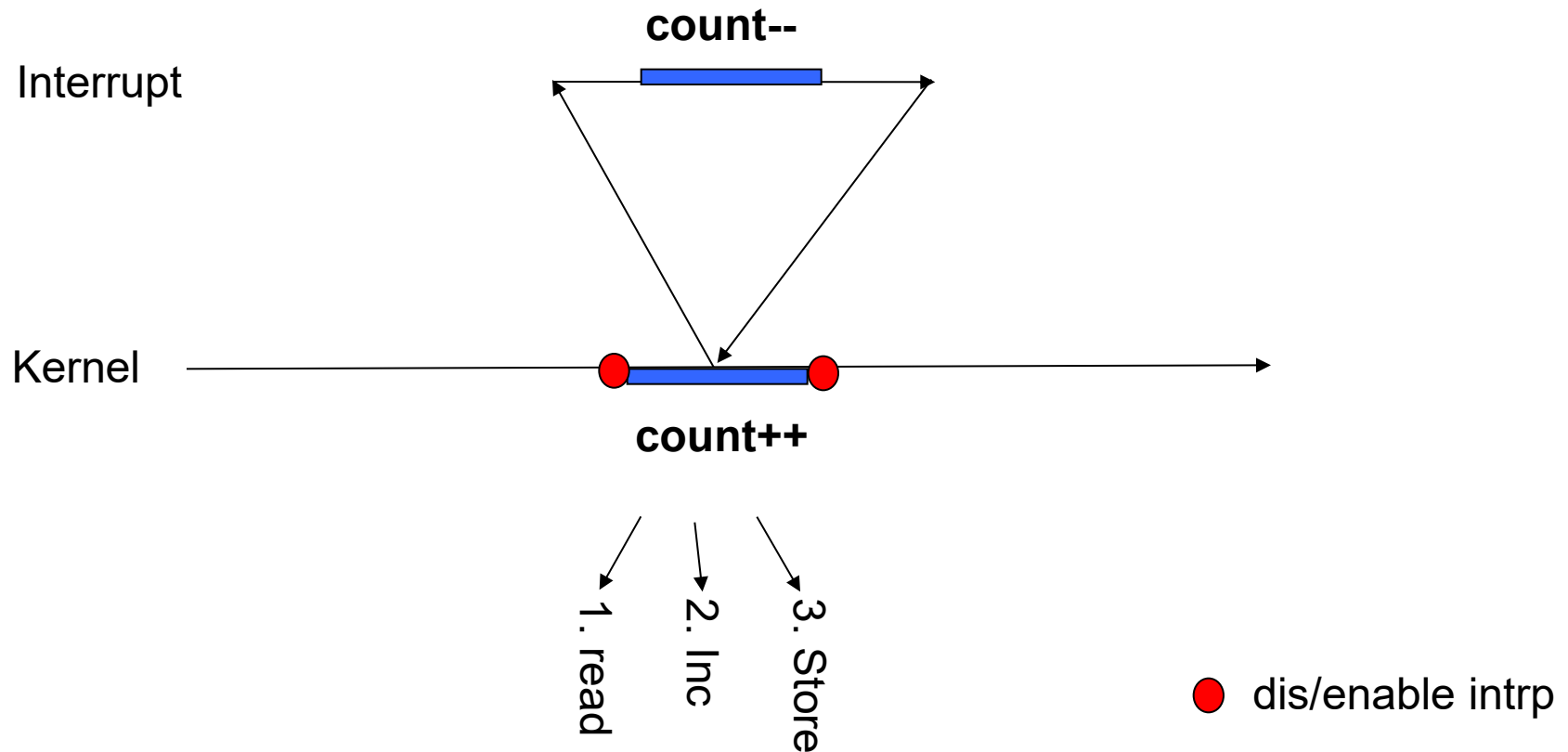


When the CS problem occurs in Operating System?

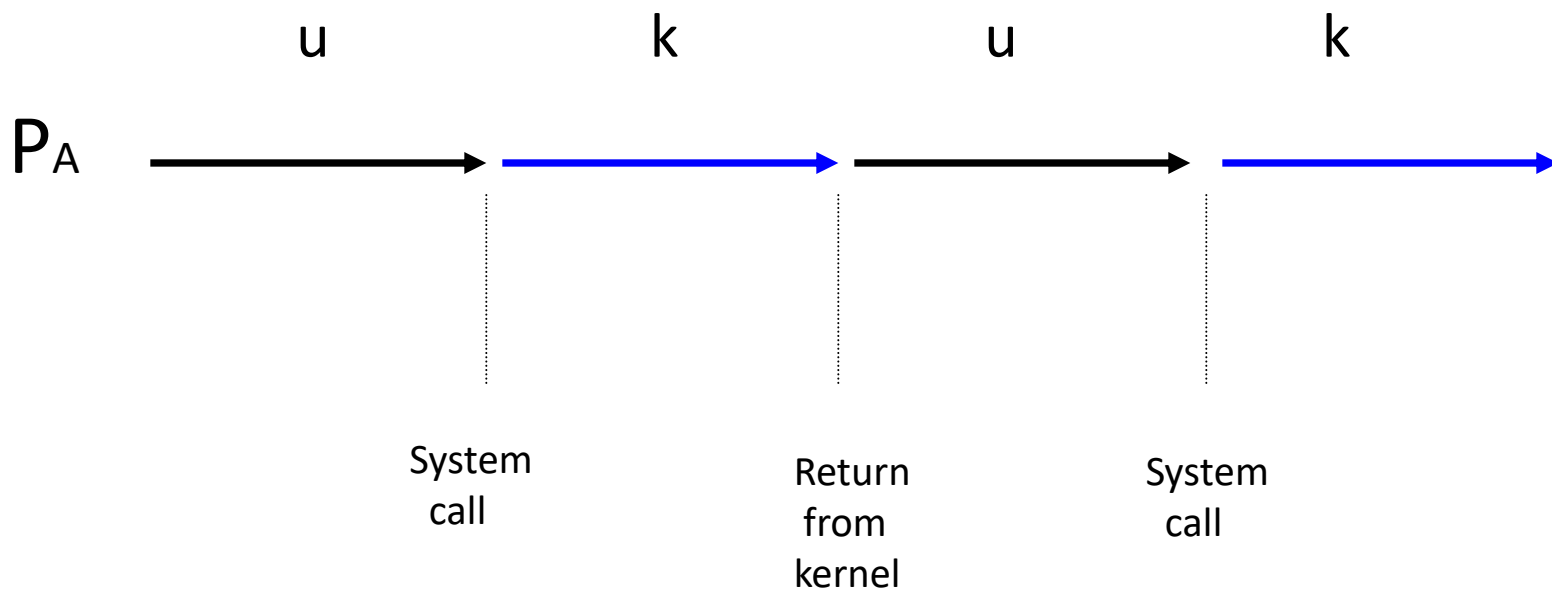
1. Between Interrupt handler and kernel
2. When context switch occurs while the kernel is performing system call
3. Multiprocessor – kernel data in the shared memory

CS problem in OS (1/3): Interrupt handler v.s. kernel

count: a kernel variable

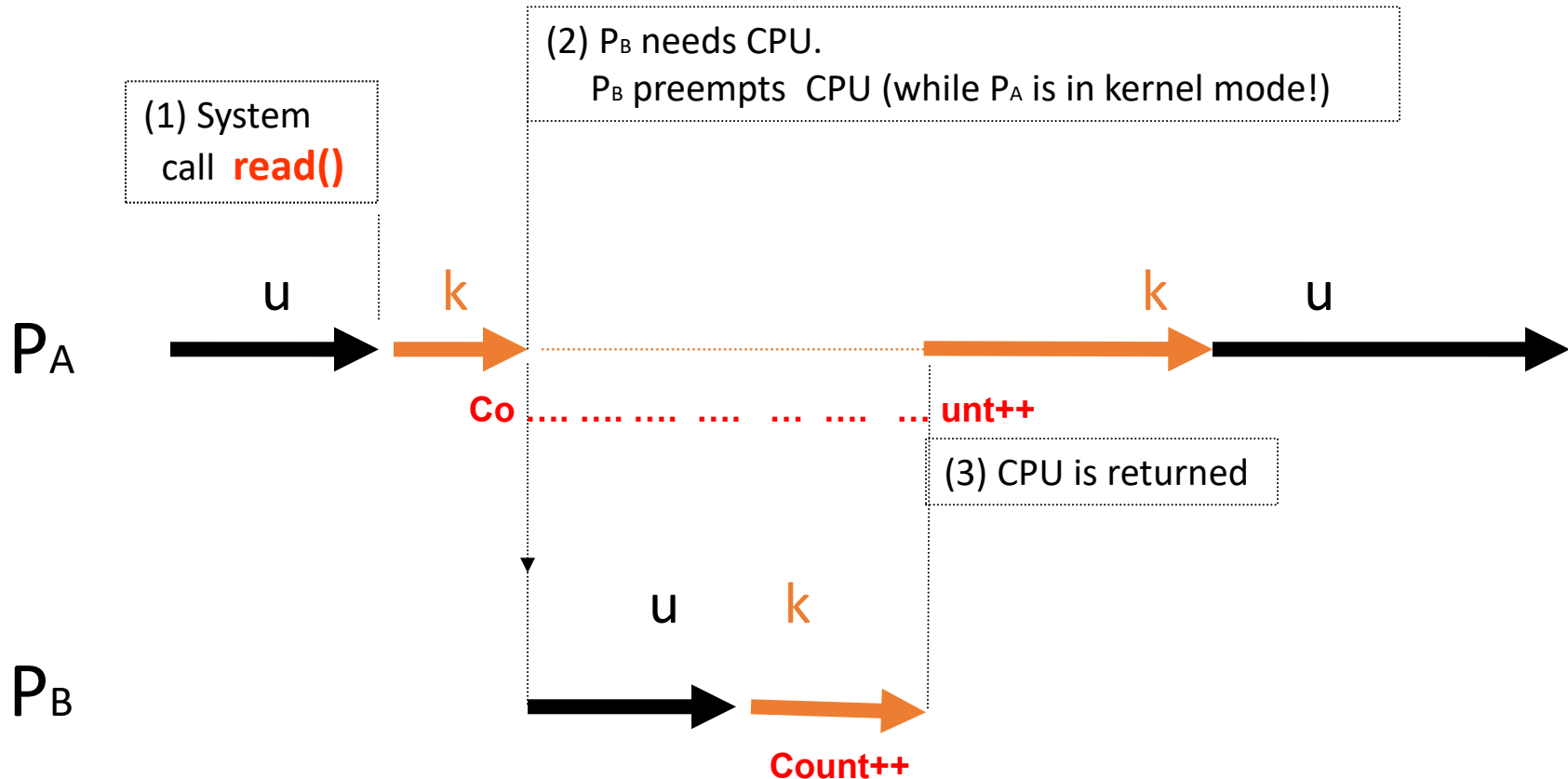


CS problem in OS (2/3): Preempt a process running in kernel?



- Even if there is no shared data between the address spaces of two processes, kernel data are accessed while system call is performed.
- Interleaved execution can occur if the CPU is preempted while accessing the kernel data

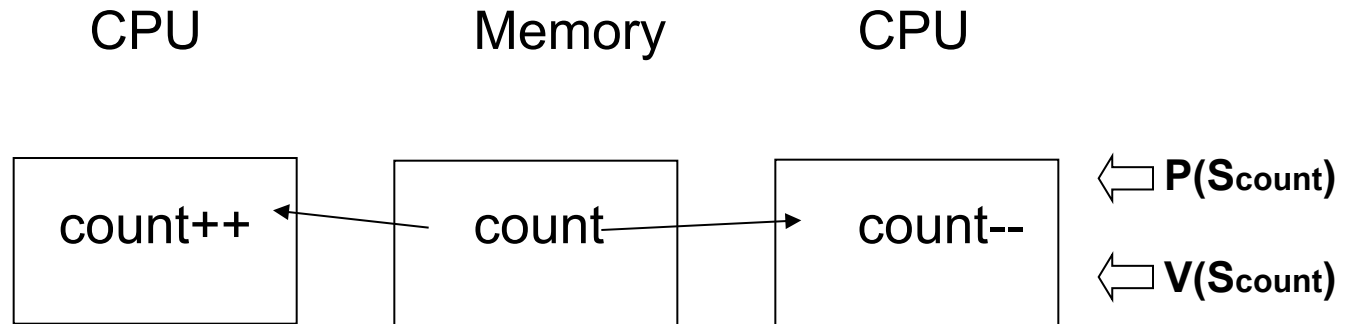
If you preempt CPU while in kernel mode



**** Do not preempt CPU if CPU was executing in kernel mode

**** UNIX waits until exit from kernel (until system call is finished)

CS problem in OS(3/3): multiprocessor



- Who stores count the last? race condition!
interrupt en/disable does not work here
- Entire kernel is one big C.S. - only one CPU can enter kernel at one time, or
- Guard each kernel shared variable with respective semaphore then kernel consists of many small critical sections